

Lab02

Matheus Rodrigues da Silva (259256)

Processamento de bases de dados

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(dplyr)
```

Criando a função ggcal

```
ggcal <- function(dates, fills) {
  data <- tibble(date = dates, fill = fills) |>
  mutate(
    year = year(date),
    month = month(date, label = TRUE, abbr = FALSE),
    weekday = wday(date, label = TRUE, abbr = TRUE, week_start = 7),
    day = day(date),
    week_of_month = ceiling((day + wday(floor_date(date, "month")) - 1) / 7)
  )
}
```

```

p <- ggplot(data, aes(x = weekday, y = week_of_month, fill = fill)) +
  geom_tile(color = "white", linewidth = 0.3) +
  facet_wrap(~ month, ncol = 4) +
  scale_y_reverse() +
  theme_minimal() +
  labs(x = NULL, y = NULL) +
  theme(
    axis.text.y = element_blank(),
    axis.ticks = element_blank(),
    axis.text.x = element_text(size = 8, color = "black"),
    panel.grid = element_blank(),
    panel.spacing = unit(0.4, "lines"),
    strip.background = element_rect(fill = "grey90", color = NA),
    strip.text = element_text(face = "bold", size = 10),
    legend.position = "bottom",
    legend.key.width = unit(1.5, "cm"),
    plot.title = element_text(face = "bold", size = 14, hjust = 0.5)
  )

return(p)
}

```

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

****R*:** A razão entre o total de vôos que atendem a esta condição e o número total de vôos que possuem informação de atraso.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

- a) Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
- b) Remova observações que tenham valores faltantes em campos de interesse;
- c) Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- d) Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
- e) A função deve receber apenas dois argumentos:
 - i. `input`: o conjunto de dados (referente ao lote em questão);

- ii. pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(dplyr)

getStats <- function(input, pos) {
  input |>
    filter(
      AIRLINE %in% c("AA", "DL", "UA", "US")
    ) |>
    filter(
      !is.na(DAY), !is.na(MONTH), !is.na(AIRLINE), !is.na(ARRIVAL_DELAY)
    ) |>
    group_by(DAY, MONTH, AIRLINE) |>
    summarise(
      total = n(),
      atrasados = sum(ARRIVAL_DELAY > 10),
      .groups = "drop"
    )
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- a. Configure o tamanho do lote (chunk) para 100 mil registros;
- b. Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
- c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa)

```
mycols = cols_only(
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)

in2 = read_csv_chunked(
  file = "flights.csv",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 100000,
```

```
col_types = mycols,  
show_col_types = FALSE)
```

4. Crie uma função chamada `computeStats` que:

- a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
- c. Cia: sigla da companhia aérea;
- ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$

```
library(lubridate)  
  
computeStats <- function(stats) {  
  stats |>  
  group_by(DAY, MONTH, AIRLINE) |>  
  summarise(  
    total = sum(total),  
    atrasados = sum(atrasados),  
    .groups = "drop"  
  ) |>  
  mutate(  
    Data = make_date(year = 2015, month = MONTH, day = DAY),  
    Cia = AIRLINE,  
    Perc = atrasados / total  
  ) |>  
  select(Cia, Data, Perc)  
}
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- a. Instale e carregue os pacotes `ggcal` e `ggplot2`.
- b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:

- d. Criar um subconjunto de stats de forma a conter informações de atraso e data apenas da Cia. Aérea dada por cia.
- ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, x representa as datas de interesse e y, os percentuais de atraso para as datas descritas em x.
- iii. Retornar para o usuário a base do calendário criada acima.
- iv. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
#library(ggcal)
library(ggplot2)

pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027",
  name = "Percentual de Atraso",
  labels = scales::percent_format(accuracy = 1)
)

baseCalendario <- function(stats, cia) {
  sub_stats <- stats |>
    filter(Cia == cia)
  p <- ggcal(sub_stats$Data, sub_stats$Perc)

  return(p)
}

resultado_final <- computeStats(in2)

cAA <- baseCalendario(resultado_final, "AA")

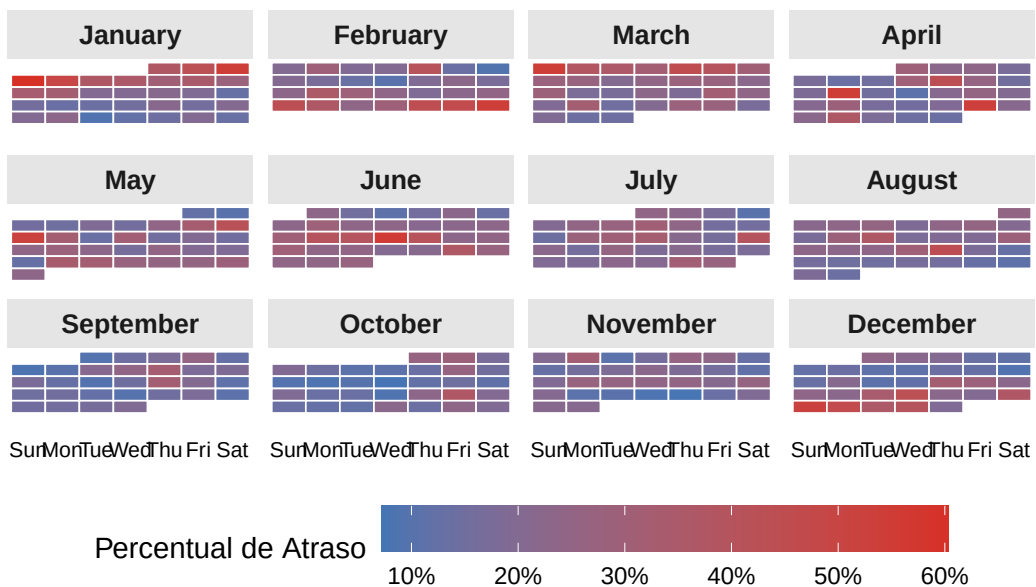
cDL <- baseCalendario(resultado_final, "DL")

cUA <- baseCalendario(resultado_final, "UA")

cUS <- baseCalendario(resultado_final, "US")
```

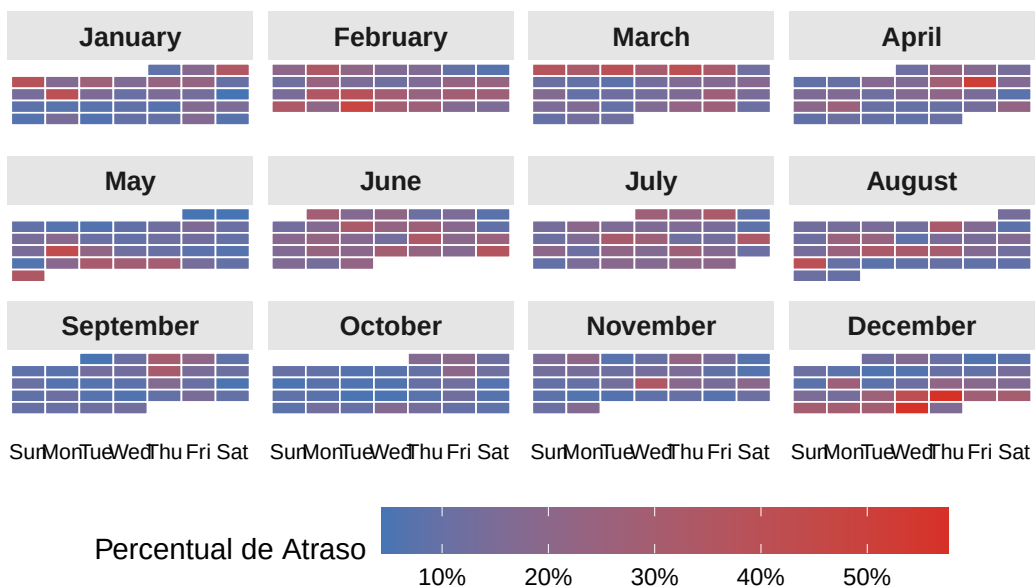
```
cAA + pal + ggtitle("Voos Atrasados - American Airlines (AA)")
```

Voos Atrasados – American Airlines (AA)



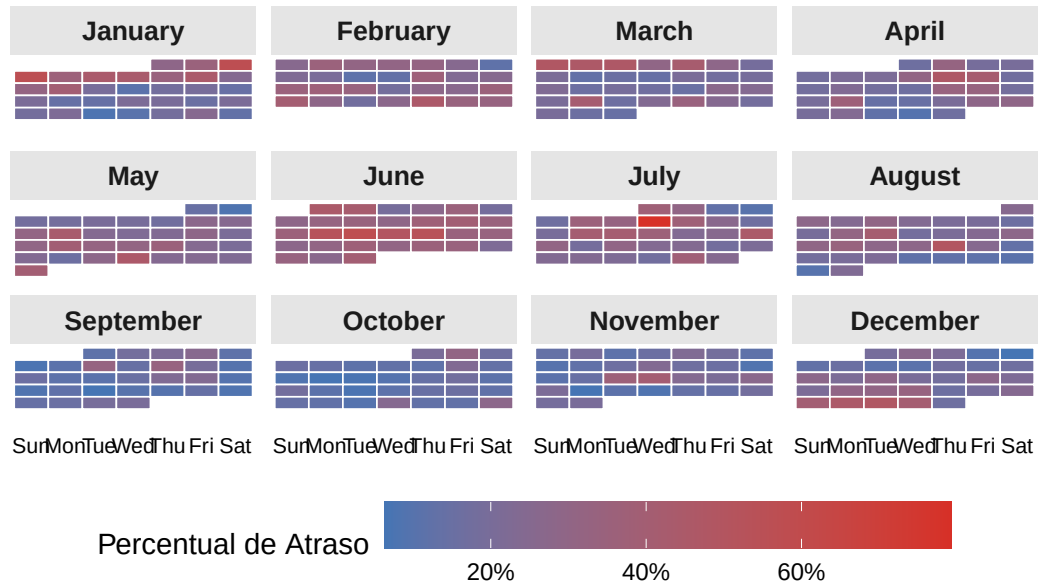
```
cDL + pal + ggtitle("Voos Atrasados - Delta Air Lines (DL)")
```

Voos Atrasados – Delta Air Lines (DL)



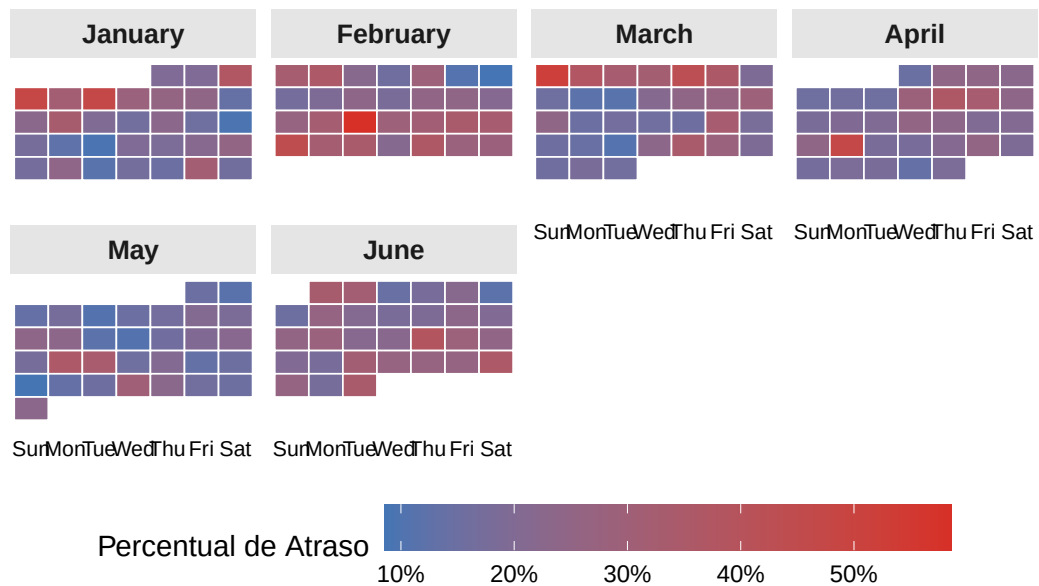
```
cUA + pal + ggtitle("Voos Atrasados - United Airlines (UA)")
```

Voos Atrasados – United Airlines (UA)



```
cUS + pal + ggtitle("Voos Atrasados - US Airways (US)")
```

Voos Atrasados – US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-23 22:09:29 -03"
```